MedOracle

Multimodal Emotional Recognition System with Explainability

Full Project Technical Report

Level 4 Final Year Research Project
Faculty of Information Technology, University of Moratuwa

Team Members
Suhira Balarajan — 214206G | Adshaya Balarajah — 214024V | Vanaiyan Kirupagaran — 214215H

Supervisor: Dr. Firdhous M.F.M.

2026

Table of Contents

1. Executive Summary

This report is the authoritative technical reference for the MedOracle Final Year Project at the Faculty of Information Technology, University of Moratuwa (2026). It documents every architectural decision, dataset strategy, mathematical formulation, evaluation protocol, and implementation step required to build, integrate, and evaluate the complete system. It is intended to be used by all three team members as the single source of truth throughout the project lifecycle.

The project builds a multimodal emotion recognition system that fuses three input streams — EEG signals, galvanic skin response (GSR), and facial video — to detect five discrete emotional states: stress, calm, happy, sad, and angry. The system is made transparent and trustworthy through SHAP-based explainability and an LLM-powered conversational interface that translates model decisions into plain-English explanations accessible to clinicians and end users.

Key Design Decisions Documented in This Report

Label harmonization strategy for DEAP (continuous V/A/D) and CREMA-D (6 classes) to 5 shared target classes.
Concrete, threshold-based signal quality criteria for all three modalities.
Full mathematical formulation of the quality-aware confidence-weighted gated fusion mechanism.
4-second temporal synchronization window with per-modality sampling specifications.
Five-condition ablation study and robustness evaluation plan.
16-week implementation timeline with integration checkpoints.

2. Project Overview

2.1 Aim and Objectives

The project aims to build a multimodal emotion recognition system that is robust, adaptable, and interpretable, combining video, EEG, and GSR to enhance the accuracy, reliability, and transparency of emotion detection in real-world conditions.

Objectives:

- Develop a multimodal emotion recognition system integrating EEG, GSR, and facial video.
- Implement bidirectional cross-modal attention between EEG and GSR signals.
- Design and implement a confidence-weighted gated fusion mechanism with graceful degradation.
- Apply Kernel SHAP to the fusion pipeline for per-modality feature importance.
- Compute a quantitative faithfulness metric to validate SHAP explanations.
- Build an LLM-powered chatbot that converts SHAP outputs to plain-English explanations.
- Develop a full-stack web application with FastAPI backend, PostgreSQL database, and React dashboard.
- Achieve subject-independent and actor-independent evaluation with macro-F1 as the primary metric.

2.2 Five Target Emotion Classes

All three members must use the same five discrete emotion classes with consistent integer encoding:

Class ID — Label	Description

0 — stress	High arousal, low valence, low dominance. Physiological: elevated sympathetic activation.
1 — calm	Low arousal, high valence. Physiological: relaxed, low sympathetic activity.
2 — happy	High arousal, high valence. Positive affect with elevated engagement.
3 — sad	Low arousal, low valence. Withdrawal-oriented negative affect.
4 — angry	High arousal, low valence, high dominance. Approach-oriented hostile affect.

Critical Consistency Requirement

The EMOTION_CLASSES dictionary {stress:0, calm:1, happy:2, sad:3, angry:4} must be identical across all three members' codebases. A shared label_harmonization.py module must be placed in a common utilities folder and imported by all members. Any divergence in encoding will cause silent errors in fusion and SHAP computation.

3. Full System Architecture

3.1 End-to-End Data Flow

The system processes inputs through a strictly sequential modular pipeline. Each stage produces a well-defined Python dictionary that serves as the input contract for the next stage.

Stage	Owner	Input	Output
Stage 1	Member 1	EEG (32ch, 128Hz) + GSR (4Hz) from DEAP	physiological_prediction_dict containing class probabilities, confidence, signal quality flags
Stage 2	Member 2	Facial video (30fps) + physiological_prediction_dict	prediction_output dict containing fused probabilities, modality weights, per-modality sub-predictions
Stage 3	Member 3	prediction_output dict	shap_output dict: SHAP values, feature importance, faithfulness score, signal reliability
Stage 4	Member 3	shap_output dict	LLM plain-English explanation via Claude/GPT-4 API
Stage 5	Member 3	All session data	FastAPI REST responses to authenticated React frontend

3.2 Architectural Layers

Layer	Technologies
ML Pipeline	Python, TensorFlow/PyTorch, MNE, NeuroKit2, OpenCV, YOLOv8, SHAP, scikit-learn
Backend API	FastAPI (Python), JWT authentication, Pydantic models, SQLAlchemy ORM
Database	PostgreSQL (production) / SQLite (development), 4 tables, Alembic migrations
LLM Integration	Anthropic Claude API or OpenAI GPT-4 API, structured prompt engineering, multi-turn history
Frontend	React.js, Recharts (data visualisation), Axios (HTTP client), CSS modules
DevOps	Git/GitHub, Docker (optional), environment variables for API keys

3.3 Design Principles

- **Modular independence:** Each member's component is independently testable via a standardised Python dictionary interface. No member's module should import directly from another member's internal code.
- **Graceful degradation:** The system continues to produce a valid emotion prediction even when one or more modalities are unavailable or flagged as poor quality. The fusion gate redistributes weight automatically.
- **Clinically transparent:** Every prediction is accompanied by a SHAP explanation and an LLM-generated plain-English summary traceable to specific physiological signals.
- **User-personalised:** All data is scoped per authenticated user via JWT tokens. No data leakage between users is permitted at the API or database level.
- **Subject-independent evaluation:** All evaluation follows subject-independent protocols (LOSO for DEAP, actor-independent k-fold for CREMA-D) to ensure results are generalisable and comparable to literature.

4. Dataset Strategy and Label Harmonization

Why This Section is Critical

Neither DEAP nor CREMA-D natively provides labels in the five target classes. DEAP uses continuous valence/arousal/dominance ratings. CREMA-D has six discrete classes that do not map cleanly to the target set. Without a rigorous, academically justified harmonization strategy, the entire system's label validity is in question. This section defines the exact mapping rules that both Member 1 and Member 2 must follow.

4.1 DEAP Dataset

Property	Value
Subjects	32 participants
Trials per subject	40 music video clips
Total trials	1,280 (estimated ~1,100 after boundary removal)
EEG channels	32 channels at 128 Hz
Peripheral signals	GSR, EMG, EOG, respiration, skin temperature at 512 Hz (downsampled to 128 Hz)
Labels	Continuous ratings: Valence (1-9), Arousal (1-9), Dominance (1-9), Liking (1-9)
Trial duration	60 seconds per trial (3s pre-trial baseline included)
Owner	Member 1 (Suhira Balarajan)

4.2 CREMA-D Dataset

Property	Value
Actors	91 actors (48 male, 43 female), ages 20-74
Sentences	12 sentences per actor per emotion
Original classes	6: ANG, DIS, FEA, HAP, NEU, SAD
Clips (original)	91 x 12 x 6 = 6,552 clips
Clips (after mapping)	~5,460 clips after dropping DIS (~17% loss)
Video format	Audio-visual clips at approximately 29.97 fps
Label format	Discrete emotion class per clip
Owner	Member 2 (Vanaiyan Kirupagaran)

4.3 DEAP: Valence-Arousal-Dominance to 5-Class Mapping

DEAP provides continuous ratings on a 1-9 scale for valence (V), arousal (A), and dominance (D). A threshold of 5.0 is applied to each dimension as the midpoint of the scale. The mapping is derived from two foundational

theoretical frameworks: Russell's (1980) Circumplex Model of Affect for the valence-arousal plane, and Mehrabian's (1996) Pleasure-Arousal-Dominance (PAD) model, which is the standard in affective computing for distinguishing stress from anger within the same quadrant.

Condition (threshold = 5.0)	Assigned Class	Academic Basis
V >= 5 AND A >= 5	happy	High valence, high arousal — Russell's excited/elated quadrant
V >= 5 AND A < 5	calm	High valence, low arousal — Russell's relaxed/serene quadrant
V < 5 AND A < 5	sad	Low valence, low arousal — Russell's depressed/lethargic quadrant
V < 5 AND A >= 5 AND D < 5	stress	Low control (D<5) differentiates stress from anger — Mehrabian PAD
V < 5 AND A >= 5 AND D >= 5	angry	High dominance (D>=5) — approach-oriented hostile state
V == 5 OR A == 5	DISCARD	Boundary samples are ambiguous — remove from training set

Citations required: Russell, J.A. (1980). A circumplex model of affect. Journal of Personality and Social Psychology, 39(6), 1161-1178. | Mehrabian, A. (1996). Pleasure-arousal-dominance: A general framework. Current Psychology, 14(4), 261-292.

4.4 CREMA-D: 6-Class to 5-Class Mapping

CREMA-D Class	Target Class	Justification
HAP (Happy)	happy	Direct mapping — identical semantic content
SAD (Sad)	sad	Direct mapping — identical semantic content
ANG (Angry)	angry	Direct mapping — identical semantic content
NEU (Neutral)	calm	Neutral state = low arousal, non-negative affect, closest to calm
FEA (Fear)	stress	Fear shares the same ANS activation profile as stress: elevated sympathetic response, high arousal, low dominance. Kreibig (2010)
DIS (Disgust)	DISCARD	No clean mapping. Disgust occupies an ambiguous position between sad and angry. Inclusion introduces label noise. Dropping ~910 clips is preferable to noisy training.

Citation for FEA->stress: Kreibig, S.D. (2010). Autonomic nervous system activity in emotion: A review. Biological Psychology, 84(3), 394-421.

4.5 Cross-Member Label Consistency

Both Member 1 and Member 2 must import their label mapping functions from a single shared file named label_harmonization.py located in a common utilities folder at the repository root. This file defines the EMOTION_CLASSES dictionary, the deap_labels_to_emotion() function, and the CREMA_D_MAPPING

dictionary. Neither member may define their own emotion class encoding. The apply_deap_mapping() and apply_cremad_mapping() functions must be the only entry points for label assignment in both modules.

Class Balance Advisory

DEAP: After V/A/D thresholding, the sad and calm classes are likely overrepresented because DEAP stimuli were selected to evoke a broad range of affect but subjects tend to rate many stimuli near the boundary. Address this with class-weighted loss during training. Do NOT use oversampling on EEG data as it introduces temporal artifacts.
CREMA-D: After dropping DIS, the five remaining classes are approximately balanced (each actor performs all five). Class weighting is still recommended for safety.

5. Member 1: Physiological Module

Owner: Suhira Balarajan (214206G) **Dataset:** DEAP **Evaluation:** LOSO cross-validation (32 folds)

5.1 Preprocessing Pipeline

Raw DEAP EEG data requires several preprocessing stages before feature extraction. The preprocessing must follow the standard DEAP protocol (Koelstra et al., 2012).

- **Baseline removal:** Subtract the per-channel mean of the 3-second pre-trial baseline period from the entire trial signal. This removes subject-specific DC offsets that would dominate frequency features.
- **Band-pass filtering:** Apply a band-pass filter between 4 Hz and 45 Hz to retain the five canonical EEG frequency bands (delta, theta, alpha, beta, gamma) and reject DC drift and high-frequency noise.
- **Artifact removal:** Apply Independent Component Analysis (ICA) to suppress eye movement (EOG) and muscle (EMG) artifacts. Flag channels with amplitude exceeding +/-100 microvolts as potentially artifactual.
- **Segmentation:** Segment each 60-second trial into 4-second non-overlapping epochs, yielding up to 15 epochs per trial. Apply a Hanning window to each epoch to reduce spectral leakage.
- **Feature extraction:** For each epoch and each channel, extract power spectral density features for the five frequency bands using Welch's method. Compute differential entropy (DE) features as log(variance) per band — DE is the dominant feature type in recent EEG emotion literature.
- **GSR preprocessing:** Apply a low-pass filter at 5 Hz. Decompose GSR into tonic (skin conductance level, SCL) and phasic (skin conductance response, SCR) components using cvxEDA or nk.eda_phasic(). Extract features: mean SCL, number of SCRs, mean SCR amplitude.

5.2 Bidirectional Cross-Modal Attention Architecture

The bidirectional cross-modal attention network allows EEG features and GSR features to mutually inform each other before joint classification. This is the primary novel contribution of Member 1's module.

Component	Specification
Input — EEG branch	Feature tensor of shape (32 channels x 5 bands) = 160-dimensional vector per epoch
Input — GSR branch	Feature vector of 6-8 handcrafted features per epoch (SCL mean, SCR count, SCR amplitude, rise time, etc.)
EEG-to-GSR attention	EEG features attend to GSR features: each EEG channel's feature is reweighted by a learned attention score computed against the GSR feature vector
GSR-to-EEG attention	GSR features attend to EEG features: GSR is conditioned on which EEG frequency bands are most activated
Fusion	Concatenate the two attention-refined representations and pass through a 2-layer MLP
Classification head	Linear layer mapping fused representation to 5 class logits, followed by softmax
Loss function	Cross-entropy with class weights (computed from training fold distribution)
Regularisation	Dropout (p=0.4) after each dense layer. L2 weight decay on all parameters.

5.3 Signal Quality: EEG and GSR Criteria

Member 1 must compute and export signal quality grades for both EEG and GSR. These grades are consumed by Member 2's fusion gate and must follow the three-tier system defined below exactly.

Modality	Criterion	Good	Degraded	Poor
EEG	Flat channels (variance < 0.5 uV^2)	0 channels	1-2 channels	> 2 channels
EEG	Amplitude out of range (> +/-100 uV)	< 5% of samples	5-15%	> 15%
EEG	NaN or Inf values present	None	None	Any present
GSR	Signal in physiological range (0.5-20 uS)	>= 95% of samples	80-95%	< 80%
GSR	Flat signal (variance < 0.01 uS^2)	No	No	Yes

The final grade for each modality is the worst-scoring criterion. For example, if EEG amplitude is 'degraded' but flat channels is 'good', the EEG quality grade is 'degraded'.

5.4 Output Contract: physiological_prediction_dict

Member 1 must export this exact dictionary structure. Member 2 will receive this dictionary and will fail at runtime if any field is missing or incorrectly typed.

Field	Type and Description
predicted_emotion	string — one of: stress, calm, happy, sad, angry
confidence	float [0,1] — softmax probability of the winning class
class_probabilities	dict — {stress: float, calm: float, happy: float, sad: float, angry: float}, values sum to 1.0
signal_quality	dict — {EEG: string, GSR: string} — each value is one of: good, degraded, poor
model_version	string — version tag for reproducibility (e.g. 'physio_v1.0')

6. Member 2: Video and Fusion Module

Owner: Vanaiyan Kirupagaran (214215H) **Dataset:** CREMA-D **Evaluation:** Actor-independent 5-fold cross-validation

6.1 Video Preprocessing Pipeline

The video pipeline processes facial video clips through a four-stage architecture: frame extraction, face detection, spatial feature extraction, and temporal classification. Audio tracks in CREMA-D clips are discarded — only the visual channel is used.

6.2 Temporal Window and Frame Sampling

All video clips are processed by uniformly sampling exactly T=16 frames across the full clip duration, regardless of clip length. This design decision is justified as follows:

- T=16 provides a genuine temporal sequence for the BiLSTM, covering expression onset, apex, and offset dynamics.
- CREMA-D clips range from 1 to 3 seconds. Uniform sampling ensures consistent sequence length across all clips without padding-induced artefacts.
- 16 is a power of 2, enabling efficient GPU batch processing.
- At a representative clip duration of 2 seconds, T=16 samples one frame every 125ms — sufficient temporal resolution for micro-expression dynamics.
- The 4-second inference window (defined in Section 8) maps to 120 raw frames at 30fps, which are uniformly subsampled to T=16 for BiLSTM input.

Clips shorter than T=16 raw frames are handled by padding with the last available frame. Clips longer than needed are uniformly subsampled. Both cases maintain sequence length = 16.

6.3 YOLO Face Detection

Aspect	Specification
Model	YOLOv8n-face — a lightweight face-specific variant of YOLOv8
Input	Each of the T=16 sampled frames (full resolution)
Output per frame	Bounding box coordinates, detection confidence score [0,1]
Multi-face policy	Take the highest-confidence detection. Discard all other detections.
No-face policy	If no face is detected in a frame, replace with a black (zero) placeholder frame. This frame contributes to the video quality assessment.
Crop and resize	Crop detected face region, resize to 224x224 pixels for ResNet50 input.
Quality output	Pass detection flags and confidence scores to compute_video_quality() to determine the video signal quality grade.

6.4 ResNet50 Feature Extraction

Aspect	Specification
Architecture	ResNet50 pretrained on ImageNet (torchvision ResNet50_Weights.IMAGENET1K_V1)
Modification	Remove the final fully-connected classification head. Use the average-pooled feature map as output.
Output dimension	2048-dimensional feature vector per frame
Fine-tuning policy	Freeze the first two residual blocks (layer1, layer2). Fine-tune the last two blocks (layer3, layer4) on CREMA-D. This adapts spatial features to facial expressions without catastrophic forgetting of low-level features.
Preprocessing	Normalise pixel values with ImageNet mean=[0.485,0.456,0.406] and std=[0.229,0.224,0.225].
Training augmentation	Per-frame: random horizontal flip, random rotation (+/-10 degrees), colour jitter (brightness 0.3, contrast 0.3), random erasing (p=0.2) to simulate occlusion.

6.5 BiLSTM Temporal Classifier

The BiLSTM models temporal dynamics of facial expression across the T=16 frame sequence. The bidirectional architecture is specifically chosen because facial expressions have both an onset trajectory (neutral to peak) and an offset trajectory (peak decay back to neutral). A unidirectional LSTM would only model onset, missing the decay pattern.

Component	Specification
Input	Sequence of T=16 ResNet50 feature vectors, shape: (batch, 16, 2048)
LSTM hidden size	256 units per direction
Directions	Bidirectional — 512 total units in combined hidden state
Layers	2 stacked LSTM layers with dropout=0.3 between layers
Output used	Final timestep hidden state — shape (batch, 512) — contains full sequence context
Classification head	LayerNorm(512) -> Dropout(0.4) -> Linear(512, 128) -> ReLU -> Linear(128, 5)
Loss function	Cross-entropy with class weights. Label smoothing (epsilon=0.1) recommended.
Optimiser	Adam with lr=1e-4 for BiLSTM parameters, lr=1e-5 for fine-tuned ResNet50 layers.
Training data	CREMA-D after CREMA-D mapping (HAP, SAD, ANG, NEU->calm, FEA->stress; DIS dropped)

6.6 Signal Quality: Video Criteria

Video signal quality is assessed from three measurable criteria computed during the YOLO detection pass over the T=16 frames. The final grade is the worst-scoring criterion (any criterion failing pulls the overall grade down).

Criterion	Good	Degraded / Poor	
Face detection rate (% of T=16 frames with a detected face)	>= 70%	40-70%	< 40%
Average YOLO confidence score across detected frames	>= 0.60	0.40-0.60	< 0.40
Average frame sharpness (Laplacian variance)	>= 100	50-100	< 50

Sharpness metric: Laplacian variance of the greyscale face crop (Pech-Pacheco et al., 2000). A blurry image has low Laplacian variance.

6.7 Gated Fusion Mechanism — Mathematical Formulation

The gated fusion combines the physiological and video probability distributions using a quality-aware confidence-weighted gate. This is the core contribution of Member 2's module.

Step 1 — Entropy-Based Confidence

For each modality m, given its class probability distribution P_m over K=5 classes, the entropy-based confidence is:

*H(P_m) = -SUM_{k=1}^{K} p_{m,k} * log(p_{m,k} + epsilon) where epsilon = 1e-8*

c_m = 1 - H(P_m) / log(K)

This gives c_m in [0,1], where 1 means maximally certain and 0 means uniform (maximum uncertainty). Entropy-based confidence is preferred over max-softmax probability because neural networks systematically overestimate confidence via raw softmax (Guo et al., 2017 — On Calibration of Modern Neural Networks, ICML 2017).

Step 2 — Quality Penalty Factor

Each modality receives a quality discount factor alpha_m based on its signal quality grade. These are fixed hyperparameters, not learned:

alpha_m = 1.0 if quality = 'good'

alpha_m = 0.5 if quality = 'degraded'

alpha_m = 0.1 if quality = 'poor'

For the physiological modality, the quality grade is taken as the worse of EEG and GSR grades: q_phys = min(q_EEG, q_GSR) where min is defined by the ordering good > degraded > poor.

Step 3 — Gate Score

*g_m = c_m * alpha_m*

Step 4 — L1 Normalisation to Modality Weights

w_m = g_m / SUM_{m'} g_{m'} if SUM g_{m'} > 0

w_m = 1 / |M| otherwise (equal-weight fallback)

L1 normalisation is used instead of softmax normalisation. Softmax always produces non-zero weights even for a completely failed modality. L1 normalisation allows a modality with a zero gate score to contribute zero weight, which is critical for graceful degradation.

Step 5 — Weighted Probability Fusion

*P_fused = SUM_{m in M} w_m * P_m*

Step 6 — Final Prediction

y_hat = argmax_k P_fused(k)

confidence_fused = max_k P_fused(k)

Worked Example
Suppose: P_phys gives stress=0.79, c_phys = 0.74, quality='good' -> alpha=1.0, g_phys = 0.74
P_video gives neutral=0.52, c_video = 0.31, quality='poor' -> alpha=0.1, g_video = 0.031
w_phys = 0.74/(0.74+0.031) = 0.96 w_video = 0.031/0.771 = 0.04
P_fused is dominated by physiological. Final prediction: stress. Confidence: ~0.84
This shows how poor video quality is almost completely discounted — the system degrades gracefully.

6.8 Late Fusion Justification

The system uses late (decision-level) fusion between physiological and video modalities. This is an architectural constraint, not merely a design preference. The following arguments justify this choice and must be cited in the report:

Argument	Explanation
Cross-dataset training constraint (primary argument)	DEAP and CREMA-D contain zero overlapping subjects. During training, no sample has both EEG signals and facial video from the same person at the same moment. Intermediate fusion requires jointly training on paired multimodal samples. Since no such paired data exists, intermediate fusion is architecturally infeasible with the chosen datasets.
Graceful degradation	With late fusion, a missing modality is handled by setting its gate score to zero and redistributing weight to the available modality. With intermediate fusion, a missing modality breaks the entire forward pass.
Heterogeneous data types	EEG (32-channel time series, 128Hz), GSR (univariate, 4Hz), and video (spatial-temporal images) are fundamentally different data types requiring different preprocessing architectures. Feature-level merging destroys modality-specific structure.
Hybrid framing	The architecture is not purely late fusion. Within the physiological module (Member 1), EEG and GSR interact via bidirectional cross-modal attention — this IS intermediate fusion at the physiological level. The hybrid framing should be stated explicitly in the report: 'Our system employs intermediate fusion within the physiological modality and late fusion at the cross-modality boundary.'
SHAP interpretability	Late fusion produces modality-level weights directly interpretable by SHAP. Intermediate fusion would yield feature-level attributions mixed across modalities, which are technically valid but clinically meaningless.

6.9 Output Contract: prediction_output Dict

Member 2 must export this exact dictionary to Member 3. All fields are mandatory.

Field	Type and Description
predicted_emotion	string — final fused prediction: one of stress, calm, happy, sad, angry
confidence	float [0,1] — max probability of P_fused (the fused distribution)
class_probabilities	dict — {stress:f, calm:f, happy:f, sad:f, angry:f} — P_fused values, sum to 1.0
modality_weights	dict — {physiological:f, video:f} — L1-normalised gate scores, sum to 1.0
signal_quality	dict — {EEG:str, GSR:str, video:str} — each value: good / degraded / poor
per_modality_predictions	dict — {physiological:{emotion:str, confidence:f}, video:{emotion:str, confidence:f}, fused:{emotion:str, confidence:f}}
model_version	string — version tag for reproducibility (e.g. 'fusion_v1.0')

7. Member 3: Explainability and Web Application

Owner: Adshaya Balarajah (214024V) **Input:** prediction_output dict from Member 2

7.1 SHAP Explainability Layer

Kernel SHAP (Lundberg and Lee, 2017 — A unified approach to interpreting model predictions, NeurIPS) is applied to the full fusion pipeline, treating each modality as a 'feature'. With M=2 effective modalities (physiological, video), SHAP evaluates all 2^2=4 coalitions: {}, {physio}, {video}, {physio, video}. For each coalition, the absent modality is replaced by its mean feature vector (background distribution), and the change in output probability is measured.

Aspect	Specification
Input to SHAP	prediction_output dict, specifically class_probabilities and modality_weights
SHAP output	shap_values: {EEG:float, GSR:float, video:float} — signed contributions to the winning class probability
feature_importance	Absolute SHAP values: {EEG:float, GSR:float, video:float}
Background set	Mean class probability vector across all training samples for each modality
Explain for	The winning class (predicted_emotion) — SHAP values sum to approximately the difference between prediction confidence and the mean baseline confidence

7.2 Faithfulness Metric

The faithfulness metric validates whether the SHAP explanation is truly reflective of the model's decision-making. It follows the standard masking-and-measuring approach:

- Rank modalities by absolute SHAP value (highest first).
- Progressively mask the top-ranked modality (replace with background mean) and re-run the fusion pipeline.
- Measure the drop in confidence for the predicted emotion after each masking step.
- Compute faithfulness_score = correlation between the rank order of SHAP values and the rank order of confidence drops.
- A score near 1.0 means the explanation is faithful — the most important modality according to SHAP causes the largest confidence drop when removed.

The faithfulness score is stored as a float [0,1] in the shap_output dict. Scores above 0.7 are considered acceptable for reporting.

7.3 LLM Chatbot Layer

The LLM chatbot bridges the gap between raw SHAP values and plain-English explanations accessible to clinicians and patients. The core mechanism is called SHAP-to-Prompt Bridging.

Aspect	Specification

LLM options	Primary: Anthropic Claude API (claude-sonnet). Fallback: OpenAI GPT-4o. Privacy note: only anonymised SHAP values and emotion labels are sent — no raw signal data.
Prompt structure	Three-part prompt: (1) System prompt defining the AI's role as an empathetic health assistant, (2) Context block with the full shap_output injected as structured text, (3) User message — either the auto-explanation trigger or the user's typed question.
Auto-explanation mode	Generated automatically after every prediction. Appears as a card in the dashboard.
Interactive Q&A; mode	User types questions (e.g. 'Why was stress detected?'). Multi-turn: each reply adds to conversation history. History is stored per session in the database.
System prompt content	Instructs the LLM to: always cite the dominant modality, explain what that modality measures in lay terms, mention signal quality caveats, use non-clinical language, and not make diagnostic claims.
Context block content	Injects: predicted_emotion, confidence, modality_weights (SHAP values), signal_quality flags, per_modality_predictions, faithfulness_score.

7.4 FastAPI Backend and Endpoints

All routes except /auth/* require a valid JWT token in the Authorization header.

Endpoint	Group	Description
POST /auth/register	Auth	Create new user account. Accepts email and password. Password hashed with bcrypt.
POST /auth/login	Auth	Authenticate user. Returns JWT access token (15min expiry) and refresh token (7 days).
POST /predict	Prediction	Accepts multimodal input, runs full pipeline, stores result in DB, returns prediction_output + shap_output.
GET /explain/{session_id}	Prediction	Fetch stored SHAP values for a previously completed session.
GET /sessions	Session	Return all sessions for the authenticated user, ordered by timestamp.
GET /sessions/{id}	Session	Return detailed session data including full SHAP breakdown and per-modality sub-predictions.
POST /chat	Chatbot	Accept user message and session_id, call LLM API with SHAP context, return response, store in chat_history.
GET /chat/history/{session_id}	Chatbot	Return all chat messages for a given session.
GET /dashboard/summary	Dashboard	Return aggregated user statistics: session count, dominant emotion, dominant modality, average confidence, emotion trend over time.

7.5 Database Schema

Table	Key Columns	Purpose

users	user_id (PK), email, password_hash, created_at, display_name	One record per registered user.
sessions	session_id (PK), user_id (FK), timestamp, predicted_emotion, confidence, modality_weights (JSON), signal_quality (JSON), class_probabilities (JSON)	One record per prediction run.
shap_logs	log_id (PK), session_id (FK), shap_values (JSON), feature_importance (JSON), faithfulness_score (float), signal_reliability (JSON)	One record per session, storing full SHAP output.
chat_history	message_id (PK), session_id (FK), user_id (FK), role (user/assistant), content (text), timestamp	One record per chat message (user or LLM).

7.6 React Frontend Dashboard

Component	Content and Behaviour
Navigation bar	App name, current user, logout button.
Stat cards (top row)	Total sessions this month, most frequent emotion, dominant modality (avg SHAP), average confidence across sessions.
Emotion trend chart	Recharts LineChart — emotion confidence per session plotted over time. Two lines: fused confidence and physiological-only confidence.
SHAP bar chart	Recharts BarChart — modality contribution (SHAP values) for the most recent session. EEG, GSR, video bars.
Session history panel	Scrollable list of past sessions: date, predicted emotion, confidence. Clicking a session loads its full detail.
Chatbot panel	Chat interface. User input box, LLM response area. Pre-populated with the auto-explanation for the current session. Supports follow-up questions.
Authentication	Login and registration pages. JWT stored in memory (not localStorage — not supported). Token refreshed automatically.

8. Temporal Synchronization Strategy

8.1 The 4-Second Trial Window

All input signals are aligned to a common 4-second trial window before any processing. This window duration is selected in accordance with established EEG emotion recognition practice (Zheng and Lu, 2015; Li et al., 2022), where 4-second epochs capture sufficient temporal context for frequency-domain feature extraction while remaining suitable for near-real-time inference.

8.2 Per-Modality Sampling Specifications

Modality	Sampling Rate	Samples per 4-second Window
EEG	128 Hz	512 samples per 4-second window, 32 channels. Baseline (3s pre-trial) subtracted before windowing.
GSR	4 Hz raw -> resampled to 128 Hz	16 raw samples -> 512 after polyphase resampling. Use scipy.signal.resample_poly(x, up=32, down=1).
Video	30 fps -> 16 frames (T)	120 raw frames per 4-second window -> uniformly subsampled to T=16 for BiLSTM input.

All modalities are aligned to a shared start timestamp. In controlled recording environments, hardware trigger signals are used to synchronise device recording start times. In software-only setups, NTP-synchronised system clocks and cross-device timestamp logging must be used.

8.3 Sliding Window for Continuous Monitoring

For continuous monitoring scenarios, the 4-second window slides with a stride of 1 second, producing one new prediction per second with 75% temporal overlap between consecutive windows. This gives the system a 1 Hz output rate, appropriate for mental health monitoring applications.

Input Validation Contract

The run_full_pipeline() function must validate inputs before processing:
EEG shape must be exactly (32, 512). Any other shape raises ValueError.
GSR shape must be exactly (512,). Raw GSR at 4Hz must be resampled before calling the pipeline.
Video frames list must contain exactly 120 elements (30fps x 4s). Pad short clips with the last frame.
Validation errors must include the exact shape received and the expected shape in the error message.

9. Evaluation Plan and Ablation Study

9.1 Primary Metrics

Accuracy must NOT be the primary reported metric. With potential class imbalance after label mapping, accuracy can be inflated by a model that favours the majority class. The primary metric is macro-averaged F1 score.

Metric	Why It's Needed
Macro-averaged F1 (PRIMARY)	Treats all 5 classes equally. Harmonic mean of precision and recall averaged across all classes without weighting by class frequency.
Per-class F1, Precision, Recall	Report individually for all 5 classes. Essential for identifying which emotions the model confuses (e.g. stress vs. angry).
Confusion matrix	5x5 matrix showing predicted vs. true class counts. Mandatory figure in results section.
Cohen's Kappa	Agreement metric that accounts for chance. More conservative and honest than accuracy alone.
Accuracy	Report as a secondary metric only. Never lead with it in tables or discussion.

9.2 Cross-Validation Strategy

Component	Protocol
Physiological module (DEAP)	Leave-One-Subject-Out (LOSO): 32 folds, one per subject. Train on 31 subjects, test on the held-out subject. This is the de facto standard for subject-independent physiological emotion recognition. Random k-fold is unacceptable — same-subject trials in both train and test sets inflate accuracy via subject-specific EEG patterns.
Video module (CREMA-D)	Actor-independent 5-fold cross-validation using GroupKFold with actor ID as the group. No actor may appear in both training and test partitions within any fold. Verify with an assertion before training.
Class weighting during training	Compute class weights from the training fold distribution using sklearn compute_class_weight('balanced'). Pass weights to the loss function. Do not oversample EEG data — temporal replication introduces artefacts.
Reporting format	Report mean macro-F1 plus/minus standard deviation across folds. Example: 'Macro-F1 = 0.72 +/- 0.06 (LOSO, 32 folds, DEAP)'

9.3 Ablation Study: Five Conditions

The ablation study is the most important table in the report. It proves that each design decision contributes measurable improvement. All five conditions must be evaluated on the same test set.

Condition	What It Tests and How

C1: Physiological only	Run Member 1's module alone. No video. Establishes physiological unimodal baseline.
C2: Video only	Run Member 2's video pipeline alone. No physiological input. Establishes video unimodal baseline.
C3: Equal-weight fusion	Fuse physiological and video with fixed w=0.5 each. Tests whether fusion helps over either modality alone.
C4: Confidence-weighted (no quality gate)	Use entropy-based confidence weights but set all alpha_m = 1.0 (ignore quality grades). Tests whether confidence weighting improves over equal weights.
C5: Quality-aware gated fusion — OURS	Full method with entropy confidence AND quality penalty factors. Tests whether quality gating improves over confidence-only weighting.

Statistical significance: Apply the Wilcoxon signed-rank test (one-tailed, alpha=0.05) comparing C5 against the best-performing baseline condition. Report p-value explicitly.

9.4 Robustness Evaluation

Three additional conditions test the system's graceful degradation behaviour:

Condition	What It Tests
C6: Video degraded (quality=poor)	Force video quality to 'poor' for all test samples. Verify that physiological modality dominates (weight > 0.9) and overall accuracy does not drop catastrophically.
C7: Video missing entirely	Replace video probability distribution with a uniform distribution (maximum entropy, minimum confidence). Verify system falls back to physiological-only effectively.
C8: Physiological degraded (quality=poor)	Force EEG and GSR quality to 'poor'. Verify that video modality picks up the weight and the system still produces a valid prediction.

9.5 Literature Baselines

Report the following baselines alongside your results to contextualise performance:

Benchmark	Method	Reference Performance
DEAP — Koelstra et al. (2012)	SVM on power spectral density features	~57% accuracy (4-class valence/arousal)
DEAP — Target	Bidirectional cross-modal attention + this project's 5-class mapping	Aim to exceed 60% macro-F1 (5-class is harder than 4-class)
CREMA-D — Cao et al. (2014)	SVM + LBP features (original 6-class)	~44% accuracy (6-class)
CREMA-D — Target	YOLO + ResNet50 + BiLSTM (5-class after DIS removal)	Aim for 70%+ macro-F1 (5-class is easier than 6-class)

Majority baseline	Always predict the most frequent class	Must be beaten by all conditions

10. Implementation Timeline

Weeks	Owner	Phase	Deliverables and Activities
Week 1-2	**ALL**	**Project setup**	Repository structure agreed. Shared utilities folder created. label_harmonization.py written and tested. DEAP and CREMA-D datasets downloaded and verified. Development environments set up.
Week 3-4	**M1**	**DEAP preprocessing**	EEG filtering, ICA artifact removal, baseline subtraction. GSR decomposition. Label mapping applied. Verify class distributions.
Week 3-4	**M2**	**CREMA-D preprocessing**	Video frame extraction pipeline. YOLO face detection integration. Label mapping applied. Dataset split into actor-independent folds.
Week 5-6	**M1**	**Feature extraction + model v1**	Differential entropy feature extraction. Initial attention network architecture. First LOSO training run.
Week 5-6	**M2**	**ResNet50 fine-tuning**	Freeze/unfreeze layers. Fine-tune on CREMA-D. Validate per-class accuracy. Augmentation pipeline.
Week 7	**ALL**	**Integration checkpoint 1**	Member 1 exports physiological_prediction_dict to a shared test harness. Member 2 validates the dict schema. Any field mismatches resolved.
Week 7-8	**M1**	**Bidirectional attention training**	Full bidirectional cross-modal attention network. LOSO cross-validation. Signal quality computation finalized.
Week 7-8	**M2**	**BiLSTM training**	Temporal classifier training. Actor-independent 5-fold CV. Signal quality computation finalized.
Week 9	**M2**	**Gated fusion implementation**	Entropy confidence, quality penalty, L1 normalization, weighted probability fusion. Unit tests for all quality grade combinations.
Week 10	**ALL**	**Integration checkpoint 2**	Member 2 exports prediction_output to a shared test harness. Member 3 validates the dict schema. Run synthetic fusion evaluation.
Week 10-11	**M3**	**SHAP + faithfulness metric**	Kernel SHAP applied to fusion pipeline. Faithfulness computation. shap_output dict assembled and validated.
Week 11-12	**M3**	**FastAPI backend**	All 9 endpoints implemented. JWT authentication. PostgreSQL schema with SQLAlchemy ORM. API tested with Postman/pytest.
Week 12-13	**M3**	**LLM chatbot integration**	Prompt builder function. Auto-explanation and Q&A; modes. Conversation history storage. API key management.
Week 13-14	**M3**	**React frontend**	Stat cards, emotion trend chart, SHAP bar chart, session history, chatbot panel. Axios integration with FastAPI.
Week 15	**ALL**	**Full system integration**	End-to-end test with real inputs. Resolve interface mismatches. Load testing of FastAPI endpoints.
Week 16	**ALL**	**Ablation study + evaluation**	Run all 5+3 ablation conditions. Compute significance tests. Compile results tables and confusion matrices.
Week 17-18	**ALL**	**Report writing**	Final report draft. Each member writes their own methodology and results section. Cross-review for consistency.
Week 19-20	**ALL**	**Viva preparation**	Presentation slides. Demo preparation. Anticipated examiner questions rehearsed.

M1 = Suhira Balarajan | M2 = Vanaiyan Kirupagaran | M3 = Adshaya Balarajah | ALL = All members

11. Step-by-Step Implementation Guide

11.1 Member 1 — Physiological Module (Suhira)

1. Step 1 — Environment setup

Install MNE-Python, NeuroKit2, PyTorch, scikit-learn, NumPy, SciPy. Clone the shared repository. Confirm label_harmonization.py is importable.

2. Step 2 — DEAP download and inspection

Download the DEAP dataset (requires data sharing agreement from QMUL). Inspect the data_preprocessed_python directory. Each subject is a dict with 'data' (40 trials x 40 channels x 8064 samples) and 'labels' (40 trials x 4 ratings).

3. Step 3 — Label mapping

Import apply_deap_mapping() from label_harmonization.py. Apply to all 32 subjects. Print class distribution. Confirm no boundary samples remain. Confirm class IDs match EMOTION_CLASSES exactly.

4. Step 4 — EEG preprocessing

For each trial: (a) Extract EEG channels 0-31. (b) Subtract 3-second pre-trial baseline mean per channel. (c) Apply bandpass filter 4-45 Hz using MNE. (d) Run ICA with 20 components, manually or automatically flag eye and muscle components. (e) Segment into 4-second epochs with no overlap. (f) Apply Hanning window per epoch.

5. Step 5 — GSR preprocessing

Extract channel 36 (GSR) from DEAP peripheral signals. Apply nk.eda_clean() then nk.eda_phasic() to decompose into tonic and phasic components. Extract per-epoch features: SCL mean, SCL std, SCR count, SCR mean amplitude, SCR rise time mean.

6. Step 6 — Signal quality computation

For each epoch: compute amplitude range check, flat channel detection, NaN detection. Assign EEG and GSR quality grades per the criteria in Section 5.3. Store in the output dict.

7. Step 7 — Feature matrix assembly

For each epoch: compute differential entropy (log variance) per EEG channel per frequency band. Stack into a (32 channels x 5 bands) = 160-dimensional vector. Concatenate GSR features. Label with the emotion class from Step 3.

8. Step 8 — Bidirectional attention network

Implement the network architecture per Section 5.2. Train with Adam (lr=1e-3), class-weighted cross-entropy, dropout=0.4. Use a learning rate scheduler (ReduceLROnPlateau).

9. Step 9 — LOSO cross-validation

Run 32 folds using the LOSO protocol. Log per-fold macro-F1. Compute mean and std. Save the best checkpoint per fold.

10. Step 10 — Export interface

Implement the predict() method that takes (eeg_window, gsr_window) and returns physiological_prediction_dict with all required fields per Section 5.4. Write unit tests that assert all fields are present and correctly typed.

11.2 Member 2 — Video and Fusion Module (Vanaiyan)

1. Step 1 — Environment setup

Install PyTorch, torchvision, OpenCV, ultralytics (YOLOv8), SciPy, scikit-learn. Download YOLOv8n-face weights. Clone shared repository. Confirm label_harmonization.py is importable.

2. Step 2 — CREMA-D download

Download from the official CREMA-D repository on GitHub. Structure: VideoFlash/ contains .flv/.mp4 clips named by ActorID_SentenceID_EmotionCode_EmotionLevel. Build a manifest CSV listing clip path, actor ID, and original emotion label.

3. Step 3 — Label mapping

Import apply_cremad_mapping() from label_harmonization.py. Apply to manifest. Remove all DIS rows. Confirm five remaining classes. Confirm class IDs match EMOTION_CLASSES exactly. Save processed manifest.

4. Step 4 — Frame extraction pipeline

For each clip: open with OpenCV, count total frames, compute uniform sample indices over [0, total_frames-1] with T=16 points, read those frames. Handle read failures by padding with black frames. Store as list of 16 BGR numpy arrays.

5. Step 5 — YOLO face detection

For each of the T=16 frames: run yolo_model(frame, verbose=False). Extract highest-confidence bounding box. Crop and resize to 224x224. Record detection flag and confidence score. Store all per-frame results for quality computation.

6. Step 6 — Video quality computation

Call compute_video_quality(frame_results). Compute face detection rate, average YOLO confidence, average Laplacian variance. Assign good/degraded/poor per criteria in Section 6.6.

7. Step 7 — ResNet50 fine-tuning

Load ResNet50 with ImageNet weights. Remove FC head. Freeze layer1 and layer2. Unfreeze layer3 and layer4. Apply train_transform augmentation per epoch. Train with Adam: lr=1e-4 for LSTM params, lr=1e-5 for unfrozen ResNet layers.

8. Step 8 — BiLSTM training

Stack T=16 ResNet50 features into (1, 16, 2048) sequence. Pass through BiLSTM (hidden=256, layers=2, bidirectional=True, dropout=0.3). Apply classification head. Train with class-weighted cross-entropy. Run actor-independent 5-fold CV.

9. Step 9 — Gated fusion implementation

Implement gated_fusion() following the mathematical formulation in Section 6.7 exactly. Unit test all eight quality grade combinations (good/degraded/poor for each of two modalities). Verify modality_weights always sum to 1.0.

10. Step 10 — Export interface

Implement run_full_pipeline(eeg, gsr, video_frames, ...) per Section 6.9. Include input validation (SynchronizedInput + validate_synchronized_input). Write unit tests asserting all output fields are present and typed correctly.

11. Step 11 — Ablation study

Implement all 5+3 ablation conditions per Section 9.3-9.4. Log results to a CSV. Compute Wilcoxon signed-rank test between C5 and best baseline.

11.3 Member 3 — Explainability and Web Application (Adshaya)

1. Step 1 — Environment setup

Install FastAPI, uvicorn, SQLAlchemy, alembic, python-jose, bcrypt, pydantic, anthropic (or openai), shap, React (via create-react-app or Vite). Set up PostgreSQL locally.

2. Step 2 — Receive and validate prediction_output

Import the prediction_output dict format from Section 6.9. Write a Pydantic model that validates all required fields. Write unit tests with synthetic dicts covering edge cases (all poor quality, missing modality, etc.).

3. Step 3 — Kernel SHAP implementation

Implement SHAP computation treating each modality as a feature. Use the fusion pipeline's predict() as the model function. Define background distribution as mean class probabilities across training samples. Compute shap_values for the winning class.

4. Step 4 — Faithfulness metric

Implement the masking-and-measuring faithfulness computation per Section 7.2. Store faithfulness_score in shap_output. Write a unit test asserting that removing the highest-SHAP modality causes a larger confidence drop than removing the lowest.

5. Step 5 — shap_output dict assembly

Assemble the enriched dict: all fields from prediction_output, plus shap_values, feature_importance, faithfulness_score, signal_reliability, explanation_ready flag.

6. Step 6 — Prompt builder function

Implement build_explanation_prompt(shap_output, user_question=None). System prompt defines role and constraints. Context block injects all shap_output fields as structured text. User message is either the auto-explain trigger or the typed question.

7. Step 7 — LLM API integration

Call the Anthropic or OpenAI API with the assembled prompt. Store the response. Implement retry logic with exponential backoff. Never send raw EEG or video data to the API — only the summary dictionary.

8. Step 8 — Database schema

Create all four tables per Section 7.5 using SQLAlchemy ORM. Run alembic init and create the initial migration. Test create/read operations for each table.

9. Step 9 — FastAPI endpoints

Implement all 9 endpoints per Section 7.4. Add JWT middleware. Test each endpoint with pytest + HTTPX. Ensure all endpoints log their session data to the database automatically.

10. Step 10 — React frontend

Build all components per Section 7.6 using functional components and React hooks. Use Recharts for all charts. Use Axios for API calls. Store JWT in memory (React state), not in localStorage.

11. Step 11 — End-to-end integration test

Run the full pipeline end-to-end with a synthetic input. Verify the prediction appears in the dashboard, the chatbot produces a coherent explanation, and the session is stored in the database.

11.4 Integration Checkpoints

Checkpoint	Requirements
Checkpoint 1 (Week 7)	Member 1 exports a physiological_prediction_dict for a synthetic input (random EEG and GSR arrays). Member 2 validates the dict schema programmatically. Any missing or incorrectly typed field must be resolved before Member 2 builds the fusion module.
Checkpoint 2 (Week 10)	Member 2 exports a prediction_output dict for a synthetic input. Member 3 validates the dict schema. Any missing or incorrectly typed field must be resolved before Member 3 builds the SHAP module.
Checkpoint 3 (Week 15)	Full end-to-end integration test with real DEAP+CREMA-D derived inputs. The complete pipeline runs from raw signals to dashboard display without runtime errors. All three members must attend and sign off.

12. Technology Stack

Category	Technology	Purpose
Deep Learning	PyTorch 2.x	Primary framework for all neural network components
EEG Processing	MNE-Python	EEG filtering, ICA, epoch extraction
GSR Processing	NeuroKit2	GSR decomposition, feature extraction
Computer Vision	OpenCV	Video frame extraction, face crop processing
Face Detection	Ultralytics YOLOv8	YOLOv8n-face for per-frame face detection
Pretrained CNN	torchvision ResNet50	Spatial feature extraction with fine-tuning
Explainability	SHAP (shap library)	Kernel SHAP for modality-level attribution
ML Utilities	scikit-learn	Class weights, cross-validation, metrics
Signal Resampling	SciPy	resample_poly for GSR upsampling
Backend API	FastAPI + Uvicorn	REST API with async support
ORM	SQLAlchemy + Alembic	Database models and schema migrations
Authentication	python-jose + bcrypt	JWT token handling and password hashing
Data Validation	Pydantic v2	Request/response schema validation
Database	PostgreSQL / SQLite	Production / development
LLM	Anthropic Claude API / OpenAI	Plain-English explanation generation
Frontend	React.js + Recharts + Axios	Dashboard, charts, API communication
Version Control	Git / GitHub	Shared repository with branch-per-member strategy
Testing	pytest + HTTPX	Unit and integration tests for all modules

13. Risk Register

Risk	Likelihood	Impact	Mitigation
DEAP class imbalance after label mapping causes biased training	High	Medium	Apply class-weighted loss. Monitor per-class F1 during training. Adjust alpha penalties if needed.
CREMA-D DIS class drop causes 17% data loss affecting coverage	Certain	Low	Acceptable and pre-documented. Explicitly acknowledged as limitation in report. 5,460 remaining clips are sufficient.
Interface contract mismatch between members causes runtime failures	Medium	High	Checkpoint 1 and 2 validation gates. Pydantic models enforce schema. Unit tests assert all field types.

LLM API rate limits or cost overruns during development	Medium	Medium	Cache LLM responses during development. Use a small CREMA-D subset for testing. Set API usage budget alerts.
DEAP's 32 subjects insufficient for generalisable deep learning	High	Medium	Acknowledged as limitation. Use LOSO CV. Apply class weighting and dropout. Compare against published DEAP baselines.
Video quality consistently poor on real-world data (vs. controlled CREMA-D)	High	Medium	Graceful degradation mechanism automatically down-weights poor video. Document this in evaluation.
ResNet50 fine-tuning overfits on CREMA-D due to small effective dataset	Medium	Medium	Augmentation pipeline (flip, jitter, erasing). Freeze early layers. Early stopping. Monitor validation loss.
FastAPI performance insufficient for real-time inference	Low	Medium	ML inference is batch (4-second window). Cache model in memory at startup. Use async endpoints.
Privacy concern: patient emotional data sent to external LLM API	Certain	Medium	Only anonymised summary statistics (SHAP values, emotion labels) are sent. Never raw signals. Document this policy.
Integration delay: one member falls behind timeline	Medium	High	Integration checkpoints enforce progress gates. Synthetic dict validators allow parallel development.

14. References

[1] S. Koelstra et al., 'DEAP: A database for emotion analysis using physiological signals,' IEEE Trans. Affect. Comput., vol. 3, no. 1, pp. 18-31, 2012.

[2] H. Cao, D. G. Cooper, M. K. Kuchinsky, et al., 'CREMA-D: Crowd-sourced Emotional Multimodal Actors Dataset,' IEEE Trans. Affect. Comput., vol. 5, no. 4, pp. 377-390, 2014.

[3] J. A. Russell, 'A circumplex model of affect,' J. Pers. Soc. Psychol., vol. 39, no. 6, pp. 1161-1178, 1980.

[4] A. Mehrabian, 'Pleasure-arousal-dominance: A general framework for describing and measuring individual differences in temperament,' Curr. Psychol., vol. 14, no. 4, pp. 261-292, 1996.

[5] S. D. Kreibig, 'Autonomic nervous system activity in emotion: A review,' Biol. Psychol., vol. 84, no. 3, pp. 394-421, 2010.

[6] S. M. Lundberg and S.-I. Lee, 'A unified approach to interpreting model predictions,' in Advances in NeurIPS, vol. 30, 2017.

[7] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, 'On calibration of modern neural networks,' in Proc. ICML, 2017, pp. 1321-1330.

[8] W.-L. Zheng and B.-L. Lu, 'Investigating critical frequency bands and channels for EEG-based emotion recognition with deep neural networks,' IEEE Trans. Auton. Ment. Dev., vol. 7, no. 3, pp. 162-175, 2015.

[9] Y. Li et al., 'A novel bi-hemispheric discrepancy model for EEG emotion recognition,' IEEE Trans. Cogn. Dev. Syst., vol. 14, no. 2, pp. 601-612, 2022.

[10] Z. Zhao, Y. Wang, G. Shen, Y. Xu, and J. Zhang, 'TDFNet: Transformer-based deep-scale fusion network for multimodal emotion recognition,' IEEE/ACM Trans. Audio, Speech, Lang. Process., vol. 31, pp. 3771-3782, 2023.

[11] J. A. Pech-Pacheco, G. Cristobal, J. Chamorro-Martinez, and J. Fernandez-Valdivia, 'Diatom autofocusing in brightfield microscopy: A comparative study,' in Proc. ICPR, vol. 3, 2000, pp. 314-317.

[12] S. Poria, E. Cambria, R. Bajpai, and A. Hussain, 'A review of affective computing: From unimodal analysis to multimodal fusion,' Inf. Fusion, vol. 37, pp. 98-125, 2017.

MedOracle — Multimodal Emotional Recognition System with Explainability | Faculty of Information Technology, University of Moratuwa | 2026 | Team: Suhira Balarajan, Adshaya Balarajah, Vanaiyan Kirupagaran | Supervisor: Dr. Firdhous M.F.M.